

Project 2 Technical Report

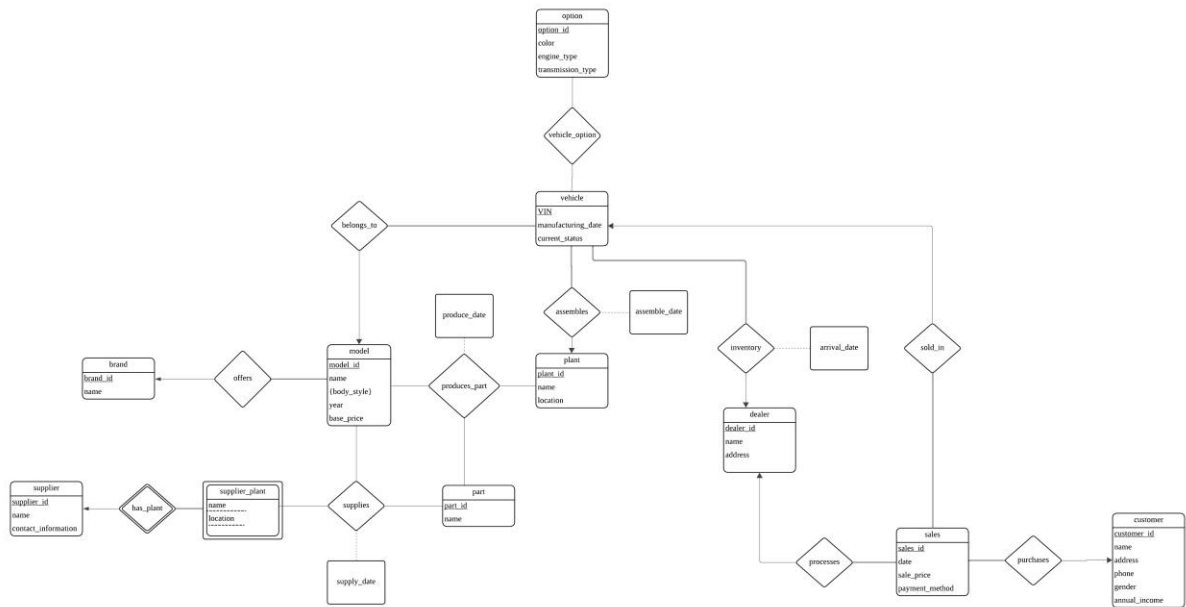
담당 교수: 정성원

학번: 20191274

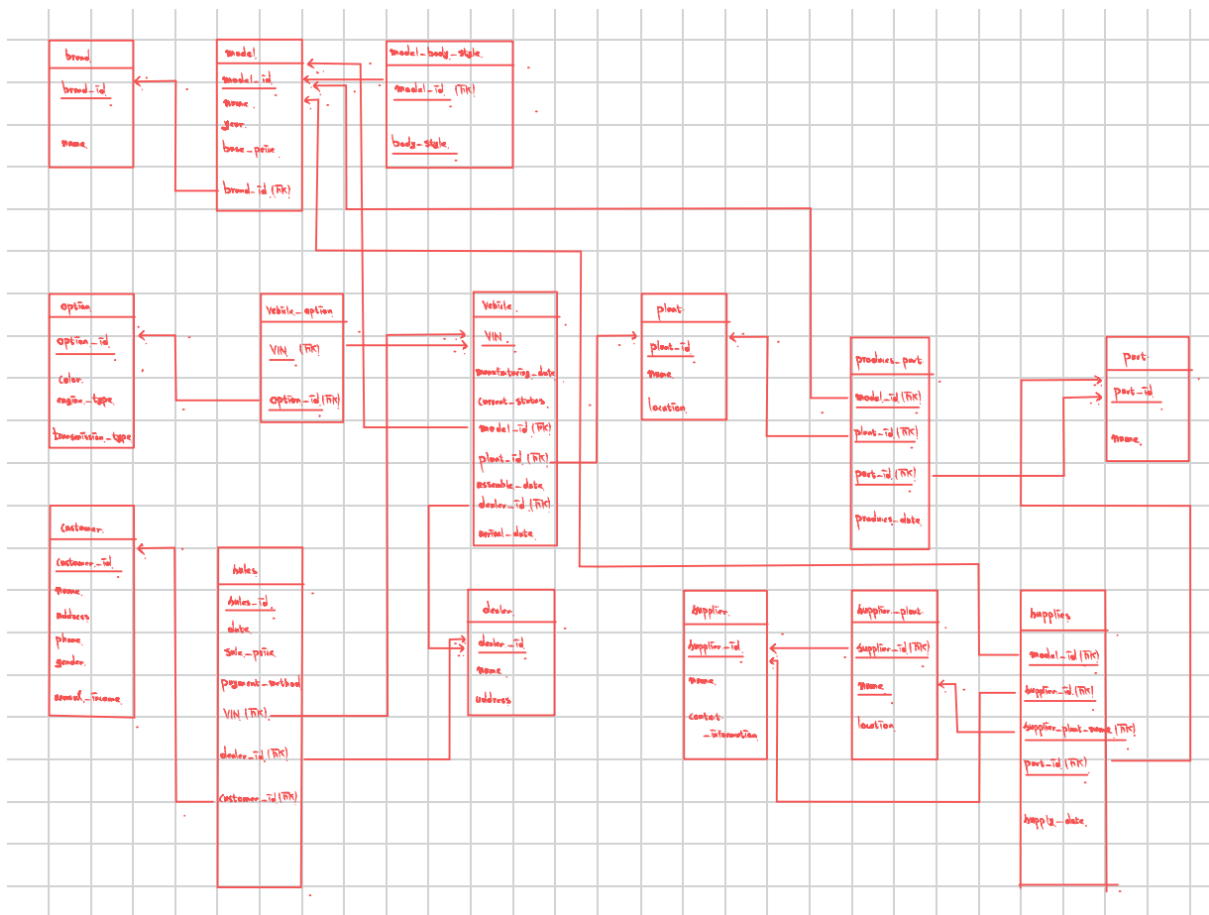
이름: 장유빈

1. Logical Schema Design

지난 Project 1에서의 E-R Diagram은 다음과 같습니다.



주어진 E-R Diagram을 바탕으로 Logical Schema를 고집어낸 결과는 다음과 같습니다.



이제, 하나하나씩 분석을 해봅시다.

먼저, brand 엔티티는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

Brand(*brand_id*, name)

여기서, *brand_id*는 PK가 될 것입니다.

그다음, model 엔티티는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

Model(*model_id*, name, year, base_price, brand_id)

여기서, *model_id*는 PK가 될 것이며, *brand_id*는 FK가 될 것입니다. 어떤 브랜드의 모델인지를 나타내주는 *brand_id*는 brand 테이블의 *brand_id*를 참조할 것입니다.

원래, model 엔티티에서는 {body_style}이라는 복합 속성을 가지고 있었습니다. 복합 속성은 PK와 함께 다른 테이블로 분리하는 것이 좋다고 우린 수업에서 배웠습니다.

따라서, model_body_style이라는 별도의 테이블로 분리하였습니다.

Model_body_style(*model_id*, body_style)

여기서 PK는 (*model_id*, body_style)이 될 것이고,

FK는 *model_id*가 될 것입니다. Model 테이블의 *model_id*를 참조할 것입니다.

그다음, option 엔티티는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

Option(*option_id*, color, engine_type, transmission_type)

여기서, option_id는 PK가 될 것입니다.

그다음, vehicle 엔티티는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

Vehicle(VIN, manufacturing_date, current_status, model_id, plant_id, assemble_date, dealer_id, arrival_date)

여기서, PK는 VIN이 될 것입니다.

또한, 어떤 모델의 차량인지를 나타내주는 model_id는 model 테이블의 model_id를 참조하는 FK가 될 것입니다.

또한, 어떤 공장에서 assemble되었는지를 나타내주는 plant_id는 plant 테이블의 plant_id를 참조하는 FK가 될 것입니다.

또한, 어떤 딜러에 의해서 차량이 보관되고 있고, 거래되었는지를 나타내주는 dealer_id는 dealer 테이블의 dealer_id를 참조하는 FK가 될 것입니다.

그다음, vehicle_option 관계는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

Vehicle_option(VIN, option_id)

여기서, (VIN, option_id)가 PK가 될 것입니다.

어떤 차량이 어떤 옵션을 가지고 있는지를 나타내줍니다.

또한, vehicle의 VIN을 참조하고 있는 VIN이 FK가 될 것입니다.

또한, option 테이블의 option_id를 참조하고 있는 option_id가 FK가 될 것입니다.

그다음, plant 엔티티는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

Plant(plant_id, name, location)

여기서, *plant_id*가 PK가 될 것입니다.

그다음, *dealer* 엔티티는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

Dealer(*dealer_id*, *name*, *address*)

여기서, *dealer_id*가 PK가 될 것입니다.

그다음, *customer* 엔티티는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

Customer(*customer_id*, *name*, *address*, *phone*, *gender*, *annual_income*)

여기서, *customer_id*가 PK가 될 것입니다.

그다음, *dealer*, *vehicle*, *customer* 사이에 Ternary relationship을 나타냈던 *sales* 관계는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

Sales(*sales_id*, *date*, *sale_price*, *payment_method*, *VIN*, *dealer_id*, *customer_id*)

여기서, *sales_id*가 PK가 될 것입니다.

어떤 차량이 판매되었는지를 나타내는 VIN은 *vehicle*의 VIN을 참조하는 FK가 될 것입니다.

어떤 딜러에 의해서 판매가 주관되었는지를 나타내는 *dealer_id*는 *dealer*의 *dealer_id*를 참조하는 FK가 될 것입니다.

또한, 어떤 고객에게 해당 차량이 판매되었는지를 나타내는 *customer_id*는 *customer*의 *customer_id*를 참조하는 FK가 될 것입니다.

그다음, 외부 공급자를 의미하는 `supplies` 엔티티는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

`Supplier(supplier_id, name, contact_information)`

여기서, `supplier_id`가 PK가 될 것입니다.

그다음, 해당 외부 공급자의 공장을 의미하는 `supplier_plant`는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

`Supplier_plant(supplier_id, name, location)`

여기서, (`supplier_id`, `name`)이 PK가 될 것입니다.

어떤 외부 공급자의 공장인지를 나타내주는 `supplier_id`는 *supplier의* `supplier_id`를 참조하는 FK가 될 것입니다.

그다음, 부품을 의미하는 `part` 엔티티는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

`Part(part_id, name)`

여기서, `part_id`가 PK가 될 것입니다.

자신의 공장에서 특정 모델을 위한 부품을 생산하고 공급하는 것을 의미하는 `produces_part`는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

`Producespart(model_id, plant_id, part_id, produces_date)`

여기서 PK는 (*model_id, plant_id, part_id*)가 될 것입니다.

어떤 공장에서 어떤 모델의 어떤 부품을 생산 및 공급하는 가를 의미합니다.

여기서, 어떤 모델을 위해 공급하는지를 나타내주는 *model_id*는 *model* 테이블의 *model_id*를 참조하는 FK가 될 것입니다.

여기서, 어떤 공장에서 해당 부품을 생산하는지를 나타내주는 *plant_id*는 *plant* 테이블의 *plant_id*를 참조하는 FK가 될 것입니다.

여기서, 어떤 부품을 생산 및 공급하는지를 나타내주는 *part_id*는 *part* 테이블의 *part_id*를 참조하는 FK가 될 것입니다.

외부 공급업체가 소유한 공장에서, 특정 모델을 위한 부품을 납품하는 관계를 의미하는 *supplies*는 다음과 같은 테이블 및 릴레이션으로 변환될 것입니다.

Supplies(model_id, supplier_id, supplier_plant_name, part_id, supply_date)

여기서, PK는 (*model_id, supplier_id, supplier_plant_name, part_id*)가 될 것입니다.

여기서, 어떤 외부 공급자의 어떤 공장인지를 나타내주는 (*supplier_id, supplier_plant_name*)는 *supplier_plant* 테이블의 (*supplier_id, supplier_plant_name*)를 참조하는 FK가 될 것입니다.

여기서, 어떤 모델을 위해 부품을 공급하는지를 나타내주는 *model_id*는 *model* 테이블의 *model_id*를 참조하는 FK가 될 것입니다.

여기서, 어떤 부품을 생산 및 공급하는지를 나타내주는 *part_id*는 *part* 테이블의 *part_id*를 참조하는 FK가 될 것입니다.

이렇게 함으로써, Logical Schema를 위의 그림과 같이 표현할 수 있었습니다.

2. Normalization Analysis

제가 만든 Logical Schema는 완벽하게 BCNF를 만족하도록 설계되었습니다.

이제, 그 이유를 하나하나씩 설명해보겠습니다.

먼저, BCNF를 만족하기 위해서는, 다음의 조건을 만족해야 합니다.

어떤 테이블의 존재하는 FD는

Trivial하거나,

$A \rightarrow P$ 에서 A 가 superkey여야 한다는 점입니다.

먼저, brand 테이블을 한번 살펴보도록 합시다.

Brand 테이블에 존재하는 FD는 $brand_id \rightarrow name$ 밖에 존재하지 않습니다.

이 FD에서는, $brand_id$ 는 superkey를 만족합니다.

따라서, brand 테이블은 BCNF를 만족하게 됩니다.

그다음, model 테이블을 한번 살펴보도록 합시다.

Model 테이블에 존재하는 FD는 $model_id \rightarrow name, year, base_price, brand_id$ 밖에 존재하지 않습니다.

이 FD에서는, $model_id$ 가 superkey입니다.

따라서, model 테이블은 BCNF를 만족하게 될 것입니다.

그다음, $model_body_style$ 테이블을 한번 살펴보도록 합시다.

이 테이블에 존재하는 FD는 $model_id, body_style \rightarrow model_id, body_style$ 뿐입니다.

이 FD는 Trivial할 것입니다.

따라서, *model_body_style* 테이블은 BCNF를 만족하게 될 것입니다.

그다음, *option* 테이블을 한번 살펴보도록 합시다.

이 테이블에 존재하는 FD는 *option_id* ~> *color*, *engine_type*, *transmission_type* 뿐입니다.

이 FD에서는, *option_id*가 superkey가 될 것입니다.

따라서, *option* 테이블은 BCNF를 만족하게 될 것입니다.

그다음, *vehicle* 테이블을 한번 살펴보도록 합시다.

이 테이블에 존재하는 FD는 VIN ~> 나머지 속성입니다.

이 FD에서는, VIN이 superkey가 될 것입니다.

따라서, *vehicle* 테이블은 BCNF를 만족하게 될 것입니다.

그다음, *vehicle_option* 테이블을 한번 살펴보도록 합시다.

이 테이블에 존재하는 FD는 VIN, *option_id* ~> VIN, *option_id* 뿐 일 것입니다.

이 FD는 Trivial할 것입니다.

따라서, *vehicle_option* 테이블은 BCNF를 만족하게 될 것입니다.

그다음, *plant* 테이블을 한번 살펴보도록 합시다.

이 테이블에 존재하는 FD는 *plant_id* ~> *name*, *location* 뿐 일 것입니다.

여기서, *plant_id*가 superkey를 만족하기 때문에,

Plant 테이블은 BCNF를 만족하게 될 것입니다.

그다음, *dealer* 테이블을 한번 살펴보도록 합시다.

이 테이블에 존재하는 FD는 *dealer_id* ~> *name*, *address* 뿐 일 것입니다.

이 FD에서는, *dealer_id*가 superkey를 만족하게 될 것입니다.

따라서, *dealer* 테이블은 BCNF를 만족하게 될 것입니다.

그다음, customer 테이블을 한번 살펴보도록 합시다.

이 테이블에 존재하는 FD는 $customer_id \sim> name, address, phone, gender, annual_income$ 뿐 일 것입니다.

여기서, customer_id는 superkey를 만족하게 될 것입니다.

따라서, customer 테이블은 BCNF를 만족하게 될 것입니다.

그다음, sales 테이블을 한번 살펴보도록 합시다.

이 테이블에 존재하는 FD는 $sales_id \sim> \text{나머지}$ 뿐 일 것입니다.

여기서, sales_id는 superkey를 만족하게 될 것입니다.

따라서, sales 테이블은 BCNF를 만족하게 될 것입니다.

그다음, supplier 테이블을 한번 살펴보도록 합시다.

여기서 존재하는 FD는 $supplier_id \sim> name, contact_information$ 뿐 일 것입니다.

이때, supplier_id는 superkey를 만족하게 될 것입니다.

따라서, supplier 테이블은 BCNF를 만족하게 될 것입니다.

그다음, supplier_plant 테이블을 한번 살펴보도록 합시다.

여기서 존재하는 FD는 $supplier_id, name \sim> location$ 뿐 일 것입니다.

그러나, supplier_id, name(공장의 이름을 의미함.)은 superkey를 만족하게 될 것입니다.

따라서, supplier_plant는 BCNF를 만족하게 될 것입니다.

그다음, part 테이블을 한번 살펴보도록 합시다.

여기서 존재하는 FD는 $part_id \sim> name$ 뿐 일 것입니다.

그러나, part_id는 superkey를 만족하게 될 것입니다.

따라서, part 테이블은 BCNF를 만족하게 될 것입니다.

그다음, produces_part 테이블을 한번 살펴보도록 합시다.

여기서 존재하는 FD는 *model_id, plant_id, part_id ~> produces_date* 뿐 일 것입니다.

그러나, 위에서 말했듯이, *model_id, plant_id, part_id*는 *superkey*를 만족하게 될 것입니다.

따라서, produces_part는 BCNF를 만족하는 테이블이 될 것입니다.

그다음, supplies 테이블을 한번 살펴보도록 합시다.

여기서 존재하는 FD는 *model_id, supplier_id, supplier_plant_name, part_id ~> supply_date* 뿐 일 것입니다.

여기서, *model_id, supplier_id, supplier_plant_name, part_id*가 우린 PK라는 것을 알고 있습니다. 따라서, *superkey*를 만족하게 될 것입니다.

따라서, supplies 테이블은 BCNF를 만족하게 될 것입니다.

이렇듯 Project 1에서 진행한 E-R Diagram을 알맞게 Logical Schema로 변환하였을 때, BCNF를 만족하는 이유는 우리가 인조 키를 두었기 때문이라고 말할 수 있습니다. 이러한 인조 키를 둬으로써, BCNF를 만족하도록 설계하는데 도움이 되었다고 말할 수 있습니다.

3. Physical Implementation

MySQL을 기반으로 하여 어떻게 Physical Schema를 구현했는지 그 과정을 설명해보도록 하겠습니다.

```
CREATE DATABASE IF NOT EXISTS auto_company;
```

```
USE auto_company;
```

먼저, auto_company라는 데이터베이스를 만들도록 하였습니다.

그리고, auto_company 데이터베이스를 사용하도록 하였습니다. |

```
-- 1. brand
CREATE TABLE brand (
    brand_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
);
```

먼저, brand 테이블에 대한 생성입니다.

PK인 *brand_id*는 *int*로 선언하였고, 자동으로 할당되도록 *auto_increment*로 설정하였습니다. 또한, primary key임을 나타내기 위해서, primary key 설정을 부여하였습니다.

브랜드의 이름을 나타내는 *name*은 *varchar(50)*으로 널널하게 선언하였고, null 값을 가질 수 없도록 *not null* 제약조건을 추가하였습니다. 또한, 브랜드명은 유일해야 하기 때문에, *unique* 제약조건을 추가하였습니다.

```
-- 2. model
CREATE TABLE model (
    model_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    year INT NOT NULL,
    base_price DECIMAL(10,2) NOT NULL CHECK
    (base_price >= 0),
    brand_id INT NOT NULL,
    FOREIGN KEY (brand_id) REFERENCES
    brand(brand_id)
);
```

그다음, model 테이블에 대한 생성입니다.

PK인 *model_id*는 *int* 타입으로 선언하였고, 자동으로 값이 할당되도록 *auto_increment*로 설정하였습니다. 또한, primary key임을 나타내기 위하여, primary key 설정을 추가하였습니다.

모델의 이름을 나타내는 *name*은 널널하게 *varchar(50)*으로 선언하였고, *not null* 제약조건을 추가하여 null 값을 가질 수 없도록 설계하였습니다.

그다음, 출시연도를 나타내는 *year*는 *int* 타입으로 선언하였고, *not null* 제약조건을 추가하였습니다.

그다음, 기본 가격을 의미하는 `base_price`는 `decimal(10, 2)`로 선언하여, 총 10자리이고, 소수점 둘째 자리수까지 가지도록 하였습니다. 또한, 가격은 `null` 값을 가질 수 없으므로, `not null` 제약조건을 추가하였습니다. 또한, 가격은 음수가 될 수 없으므로, `check (base_price >= 0)` 추가조건까지 설정하였습니다.

그다음 어떤 브랜드의 모델인지를 나타내주는 `brand_id`는 `int` 타입으로 선언하였고, `not null` 제약조건을 추가하였습니다. FK로써, 반드시 있어야 하는 값이기 때문입니다.

그다음, 위에서 말했듯이 어떤 브랜드의 모델인지를 나타내주는 `brand_id`에 대한 왜래키 제약조건을 다음과 같이 설정하였습니다.

```
FOREIGN KEY (brand_id) REFERENCES brand(brand_id)
```

```
-- 3. model_body_style
CREATE TABLE model_body_style (
    model_id INT NOT NULL,
    body_style VARCHAR(30) NOT NULL,
    PRIMARY KEY (model_id, body_style),
    FOREIGN KEY (model_id) REFERENCES
model(model_id)
);
```

그다음, `model_body_style`에 대한 테이블 생성입니다.

`Model_id`는 `int` 타입으로 선언하였고, `not null` 제약조건을 가지도록 하였습니다.

`Body_style`은 널널하게 `varchar(30)`으로 선언하였고, `not null` 제약조건을 가지도록 하였습니다.

PK인 (`model_id`, `body_style`)을 표현하기 위해서,

`PRIMARY KEY (model_id, body_style)` 제약조건을 추가하였습니다.

어떤 모델의 바디 스타일인지를 나타내주는 `model_id` FK에 대한 제약조건을 다음과 같이 추가하였습니다.

```
FOREIGN KEY (model_id) REFERENCES model(model_id)
```

그다음, option 테이블에 대한 생성입니다.

```
-- 4. option
CREATE TABLE `option` (
  option_id INT AUTO_INCREMENT PRIMARY KEY,
  color VARCHAR(30) NOT NULL,
  engine_type VARCHAR(50) NOT NULL,
  transmission_type VARCHAR(50) NOT NULL
);
```

먼저, option_id는 int 타입으로 선언하였고, 자동으로 값이 할당되도록 auto_increment 설정을 추가하였습니다. 그다음, primary key를 나타내기 위해서, primary key 제약조건을 추가하였습니다.

Color는 널널하게 varchar(30)으로 선언하였고, not null 제약 조건을 가지도록 하였습니다.

Engine_type은 널널하게 마찬가지로 varchar(50)으로 선언하였고, not null 제약 조건을 가지도록 하였습니다.

Transmission_type도 널널하게 varchar(50)으로 선언하였고, not null 제약조건을 가지도록 하였습니다.

```
-- 5. plant
CREATE TABLE plant (
  plant_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  location VARCHAR(100) NOT NULL
);
```

그다음, plant 테이블에 대한 생성입니다.

Plant_id는 int 타입으로 선언하였고, 자동으로 값이 할당되도록 auto_increment 설정을 추가하였습니다. 또한, PK임을 나타내기 위해서, primary key 조건을 추가하였습니다.

공장의 이름을 나타내주는 name은 널널하게 varchar(50)로 선언하였고, null 값을 가지면 안되기 때문에, not null 제약조건을 추가하였습니다.

공장의 위치를 나타내주는 location은 널널하게 varchar(100)으로 선언하였고, null 값을

가지면 안되기 때문에, not null 제약조건을 추가하였습니다.

```
-- 6. dealer
CREATE TABLE dealer (
  dealer_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  address VARCHAR(100) NOT NULL
);
```

그다음, dealer 테이블에 대한 생성입니다.

먼저, *dealer_id*는 *int* 타입으로 선언하였고, 자동으로 값이 할당되도록 *auto_increment* 설정을 추가하였습니다. 또한, PK임을 나타내기 위해서, *primary key* 조건을 추가하였습니다.

딜러의 이름을 나타내주는 *name*은 널널하게 *varchar(50)*으로 선언하였고, *null* 값을 가지면 안되기 때문에, *not null* 제약조건을 추가하였습니다.

딜러의 주소를 의미하는 *address*는 널널하게 *varchar(100)*으로 선언하였고, *null* 값을 가지면 안되기 때문에, *not null* 제약조건을 추가하였습니다.

```
-- 7. vehicle
CREATE TABLE vehicle (
  VIN VARCHAR(17) PRIMARY KEY, -- 국제 표준 차대번호 길이는 17자리
  manufacturing_date DATE NOT NULL,
  current_status VARCHAR(20) DEFAULT 'Inventory', -- 상태 기본값
  model_id INT NOT NULL,
  plant_id INT,
  assemble_date DATE,
  dealer_id INT,
  arrival_date DATE,
  FOREIGN KEY (model_id) REFERENCES
model(model_id),
  FOREIGN KEY (plant_id) REFERENCES plant(plant_id),
  FOREIGN KEY (dealer_id) REFERENCES
dealer(dealer_id)
);
```


그다음, vehicle에 대한 테이블 생성입니다.

먼저, 국제 표준 차대번호 길이는 17자리입니다.

따라서, VIN은 varchar(17)로 선언하였고, PK임을 나타내기 위해서, primary key 조건을 추가하였습니다.

제조일자를 나타내는 manufacturing_date는 DATE 타입으로 선언하였고, null 값을 가지면 안되기 때문에, not null 제약조건을 추가하였습니다.

현재 차량의 상태를 의미해주는 current_status는 널널하게 varchar(20)으로 선언하였고, default 값으로 'Inventory'를 설정하였습니다. 아직 팔리지 않고, 재고로 남아있다는 의미입니다.

어떤 모델의 차량인지를 나타내주는 model_id는 int 타입과 not null 제약조건을 가지도록 하였습니다.

어떤 공장에서 assemble되었는지를 나타내주는 plant_id는 int 타입으로 선언하였습니다.

조립일자를 나타내주는 assemble_date는 DATE 타입으로 선언하였습니다.

어떤 딜러에 의하여 해당 차량이 관리되고 있는지를 나타내주는 dealer_id는 int 타입으로 선언하였습니다.

차량 입고일을 의미하는 arrival_date는 DATE 타입으로 선언하였습니다.

어떤 모델의 차량인지를 나타내주는 model_id FK에 대한 왜래키 제약조건을 다음과 같이 나타내었습니다.

```
FOREIGN KEY (model_id) REFERENCES model(model_id)
```

해당 차량이 어떤 공장에서 assemble되었는지를 나타내주는 plant_id FK에 대한 왜래키 제약조건은 다음과 같이 나타내었습니다.

```
FOREIGN KEY (plant_id) REFERENCES plant(plant_id)
```

어떤 딜러에 의하여 해당 차량이 관리되고 있는지를 나타내주는 dealer_id FK에 대한 왜래키 제약조건은 다음과 같이 나타내었습니다.

FOREIGN KEY (dealer_id) REFERENCES dealer(dealer_id)

```
-- 8. vehicle_option
CREATE TABLE vehicle_option (
  VIN VARCHAR(17) NOT NULL,
  option_id INT NOT NULL,
  PRIMARY KEY (VIN, option_id),
  FOREIGN KEY (VIN) REFERENCES vehicle(VIN),
  FOREIGN KEY (option_id) REFERENCES `option`(option_id)
);
```

그다음, vehicle_option 테이블에 대한 생성입니다.

VIN은 varchar(17)로 선언하였고, not null 제약조건을 가지도록 하였습니다.

Option_id는 int 타입으로 선언하였고, not null 제약조건을 가지도록 하였습니다.

PK인 (VIN, option_id)에 대한 조건은 위와 같이 선언하였습니다.

어떤 차량에 대한 옵션인지를 나타내주는 VIN FK에 대한 왜래키 제약조건은 다음과 같이 선언하였습니다.

FOREIGN KEY (VIN) REFERENCES vehicle(VIN)

해당 차량이 어떤 옵션을 가지고 있는지를 나타내주는 option_id FK에 대한 왜래키 제약 조건은 다음과 같이 선언하였습니다.

FOREIGN KEY (option_id) REFERENCES `option`(option_id)

그다음, customer 테이블에 대한 생성입니다.

```
-- 9. customer
CREATE TABLE customer (
  customer_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  address VARCHAR(100),
  phone VARCHAR(20),
  gender ENUM('Male', 'Female', 'Other'), -- ENUM을 통한 제약
  annual_income DECIMAL(12,2) CHECK (annual_income >= 0)
);
```

먼저, *customer_id*는 *int* 타입으로 선언하였고, 자동으로 값이 할당되도록 *auto_increment* 설정을 추가하였습니다. PK임을 나타내기 위하여, *primary key* 조건을 추가하였습니다.

고객의 이름을 나타내주는 *name*은 널널하게 *varchar(50)*으로 선언하였고, 이름은 *null* 값을 가지면 안되기 때문에, *not null* 제약조건을 추가하였습니다.

고객의 주소를 의미하는 *address*는 널널하게 *varchar(100)*으로 선언하였습니다.

고객의 휴대전화 번호를 의미하는 *phone*은 널널하게 *varchar(20)*으로 선언하였습니다.

고객의 성별을 의미하는 *gender*는 *ENUM('Male', 'Female', 'Other')*로 제한을 걸어두었습니다. 즉, *ENUM*을 통해 제약을 둔 것입니다.

고객의 연소득을 의미하는 *annual_income*은 *decimal(12, 2)* 타입으로 선언하였습니다. 또한, 고객의 연소득은 음수일 수 없기 때문에,

*CHECK (annual_income >= 0)*와 같은 제약조건을 추가하였습니다.

```
-- 10. sales
CREATE TABLE sales (
  sales_id INT AUTO_INCREMENT PRIMARY KEY,
  date DATE NOT NULL,
  sale_price DECIMAL(12,2) NOT NULL CHECK (sale_price >= 0),
  payment_method VARCHAR(30),
  VIN VARCHAR(17) NOT NULL UNIQUE, -- 차량 1대는 1번만 팔릴 수 있으므로 UNIQUE
  dealer_id INT NOT NULL,
  customer_id INT NOT NULL,
  FOREIGN KEY (VIN) REFERENCES vehicle(VIN),
  FOREIGN KEY (dealer_id) REFERENCES dealer(dealer_id),
  FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
);
```

해당 차량의 판매기록을 기록하는 *sales* 테이블에 대한 생성은 다음과 같습니다.

먼저, *sales_id*는 *int* 타입으로 선언하였고, *auto_increment*를 통하여 자동으로 값이 할당 되도록 하였습니다. 또한, PK임을 나타내기 위해서, *primary key* 조건을 추가하였습니다.

판매일자를 의미하는 *date*은 *DATE* 타입을 사용하였고, *null* 값을 가질 수 없으므로, *not null* 제약조건을 추가하였습니다.

판매가격을 의미하는 *sale_price*는 *decimal(12, 2)*로 선언하였고, *null* 값을 가질 수 없으므로, *not null* 제약조건을 추가하였습니다. 또한, 판매가격은 음수일 수 없기 때문에, *CHECK (sale_price >= 0)*와 같은 조건을 추가하였습니다.

구매 방법을 의미하는 *payment_method*는 널널하게 *varchar(30)*으로 선언하였습니다.

어떤 차량을 판매했는지를 기록하는 *VIN*은 *varchar(17)*로 선언하였고, *not null* 제약조건과 *unique* 제약조건을 추가하였습니다. 차량 1대는 1번만 팔릴 수 있다고 가정한 셈입니다.

어떤 딜러에 의하여 해당 차량의 판매가 주관되었는지를 나타내주는 *dealer_id*는 *int* 타입으로 선언하였고, *null* 값을 가질 수 없기 때문에, *not null* 제약조건을 추가하였습니다.

어떤 고객에게 해당 차량을 판매하였는지를 나타내주는 *customer_id*는 *int* 타입으로 선언하였고, *null* 값을 가질 수 없기 때문에, *not null* 제약조건을 추가하였습니다.

어떤 차량(*VIN*)을 어떤 딜러(*dealer_id*)에 의하여 어떤 고객(*customer_id*)에게 판매했는지에 대한 FK에 대한 왜래키 제약조건은 다음과 같이 선언하였습니다.

```
FOREIGN KEY (VIN) REFERENCES vehicle(VIN),
```

```
FOREIGN KEY (dealer_id) REFERENCES dealer(dealer_id),
```

```
FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
```

```
-- 11. supplier
CREATE TABLE supplier (
  supplier_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  contact_information VARCHAR(100)
);|
```

외부 부품 공급자를 의미하는 *supplier* 테이블에 대한 생성은 다음과 같습니다.

먼저, `supplier_id`는 `int` 타입으로 선언하였고, 자동으로 값이 할당되도록 `auto_increment` 설정을 추가하였습니다. 또한, PK임을 나타내기 위해서, `primary key` 조건을 추가하였습니다.

외부 공급업체의 이름을 의미하는 `name`은 널널하게 `varchar(50)`으로 선언하였고, `null` 값을 가질 수 없기 때문에, `not null` 제약조건을 추가하였습니다.

외부 공급업체의 연락처를 의미하는 `contact_information`은 널널하게 `varchar(100)`으로 선언하였습니다.

```
-- 12. supplier_plant
CREATE TABLE supplier_plant (
  supplier_id INT NOT NULL,
  name VARCHAR(50) NOT NULL,
  location VARCHAR(100),
  PRIMARY KEY (supplier_id, name),
  FOREIGN KEY (supplier_id) REFERENCES supplier(supplier_id) ON DELETE CASCADE
);
```

외부 공급업체가 소유한 공장을 의미하는 `supplier_plant` 테이블에 대한 생성은 다음과 같습니다.

먼저, `supplier_id`는 `int` 타입과 `not null` 제약조건을 가지도록 하였습니다.

공장의 이름을 의미하는 `name`은 널널하게 `varchar(50)`으로 선언하였고, `null` 값을 가지면 안되기 때문에, `not null` 제약조건을 추가하였습니다.

공장의 위치를 의미하는 `location`은 널널하게 `varchar(100)`으로 선언하였습니다.

PK에 대한 제약조건은 다음과 같습니다.

```
PRIMARY KEY (supplier_id, name)
```

어떤 외부 공급업체가 소유한 공장의 이름에 대한 정보를 의미합니다.

어떤 공급업체가 소유한 공장인지를 나타내주는 `supplier_id` FK에 대한 왜래키 제약조건은 다음과 같습니다.

FOREIGN KEY (supplier_id) REFERENCES supplier(supplier_id) ON DELETE CASCADE

다음은 부품을 의미하는 part 테이블에 대한 생성입니다.

```
-- 13. part
CREATE TABLE part (
  part_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL UNIQUE
);
```

Part_id는 int 타입으로 선언하였고, 자동으로 값이 할당되도록 auto_increment 설정을 추가하였습니다. 또한, PK임을 나타내기 위해서, primary key 조건을 추가하였습니다.

부품의 이름을 나타내주는 name은 널널하게 varchar(50)으로 선언하였습니다. 또한, null 값을 가지면 안되기 때문에, not null 제약조건과 부품의 이름은 유일해야 하기 때문에, unique 제약조건까지 추가하였습니다.

```
-- 14. produces_part (내부 생산)
CREATE TABLE produces_part (
  model_id INT NOT NULL,
  plant_id INT NOT NULL,
  part_id INT NOT NULL,
  produce_date DATE NOT NULL,
  PRIMARY KEY (model_id, plant_id, part_id),
  FOREIGN KEY (model_id) REFERENCES model(model_id),
  FOREIGN KEY (plant_id) REFERENCES plant(plant_id),
  FOREIGN KEY (part_id) REFERENCES part(part_id)
);
```

그다음, 내부 생산을 의미해주는 produces_part 테이블에 대한 생성은 다음과 같습니다.

어떤 모델의 부품인지를 나타내주는 model_id는 int 타입과 not null 제약조건을 가지도록 하였습니다.

어떤 공장에서 해당 부품이 생산되는지를 나타내주는 plant_id는 int 타입과 not null 제약조건을 가지도록 하였습니다.

어떤 부품을 생산하는지를 나타내주는 `part_id`는 `int` 타입과 `not null` 제약조건을 가지도록 하였습니다.

해당 부품에 대한 생산일자를 의미해주는 `produce_date`는 `DATE` 타입과 `not null` 제약조건을 가지도록 하였습니다.

그다음, (`model_id`, `plant_id`, `part_id`)에 대한 PK 조건을 추가하였습니다.

그다음, 어떤 공장(`plant_id`)에서 어떤 모델(`model_id`)의 어떤 부품(`part_id`)을 생산하는지에 대한 FK에 대한 왜라키 제약조건은 다음과 같이 나타내었습니다.

```
FOREIGN KEY (model_id) REFERENCES model(model_id),
```

```
FOREIGN KEY (plant_id) REFERENCES plant(plant_id),
```

```
FOREIGN KEY (part_id) REFERENCES part(part_id)
```

```
-- 15. supplies (외부 조달)
CREATE TABLE supplies (
  model_id INT NOT NULL,
  supplier_id INT NOT NULL,
  supplier_plant_name VARCHAR(50) NOT NULL,
  part_id INT NOT NULL,
  supply_date DATE NOT NULL,
  PRIMARY KEY (model_id, supplier_id, supplier_plant_name, part_id),
  FOREIGN KEY (model_id) REFERENCES model(model_id),
  FOREIGN KEY (supplier_id, supplier_plant_name) REFERENCES supplier_plant(supplier_id, name),
  FOREIGN KEY (part_id) REFERENCES part(part_id)
);
```

그다음, 외부 조달을 의미하는 `supplies` 테이블에 대한 생성은 다음과 같습니다.

어떤 모델의 부품을 생산할 것인지를 나타내주는 `model_id`는 `int` 타입과 `not null` 제약조건을 가지도록 하였습니다.

어떤 외부 공급업체인지를 나타내주는 `supplier_id`는 `int` 타입과 `not null` 제약조건을 가지도록 하였습니다.

해당 외부 공급업체의 어떤 공장인지를 나타내주는 `supplier_plant_name`은 널널하게 `varchar(50)`으로 선언하였고, `not null` 제약조건을 가지도록 하였습니다.

또한, 어떤 부품을 공급하는지에 대해서 나타내주는 `part_id`는 `int` 타입과 `not null` 제약조건을 가지도록 하였습니다.

해당 부품의 공급일자를 의미해주는 `supply_date`는 `DATE` 타입과 `not null` 제약조건을 가

지도록 하였습니다.

PK에 대한 조건은 위와 같이 선언하였습니다.

어떤 외부 공급업체(*supplier_id*)의 어떤 공장(*supplier_plant_name*)에서 어떤 모델(*model_id*)을 위하여 어떤 부품(*part_id*)을 공급하는지에 대한 FK에 대한 왜래키 제약조건은 다음과 같이 선언하였습니다.

```
FOREIGN KEY (model_id) REFERENCES model(model_id),  
  
FOREIGN KEY (supplier_id, supplier_plant_name) REFERENCES  
supplier_plant(supplier_id, name),  
  
FOREIGN KEY (part_id) REFERENCES part(part_id)
```

추가로 7개의 Query를 보다 조금 더 빨리 실행하기 위하여 인덱스를 설정해보았습니다.

```
-- Query 1, 3, 4: 기간별(Date) 매출 조회를 위한 인덱스  
CREATE INDEX idx_sales_date ON sales(date);  
  
-- Query 2: 특정 기간의 부품 공급 기록 조회를 위한 인덱스  
CREATE INDEX idx_supplies_date ON supplies(supply_date);  
  
-- Query 5: 특정 바디 스타일(ex: convertible) 검색을 위한 인덱스  
CREATE INDEX idx_body_style ON model_body_style(body_style);
```

추가로 해당 차량이 판매되면 자동으로 *current_status*를 'Sold' 상태로 업데이트 하기 위하여 트리거 설정을 추가적으로 해보았습니다.

즉, 해당 차량이 판매되면 *vehicle* 테이블의 *current_status*를 자동으로 'Sold'로 업데이트 하도록 하였습니다.


```

DELIMITER //

CREATE TRIGGER after_sales_insert
AFTER INSERT ON sales
FOR EACH ROW
BEGIN
    -- 차량이 판매되면 vehicle 테이블의 current_status를 자동으로 'Sold'로 업데이트
    UPDATE vehicle
    SET current_status = 'Sold'
    WHERE VIN = NEW.VIN;
END; //

DELIMITER ;

```

추가로 다음과 같이 샘플링 데이터를 만들어보았습니다.

USE auto_company;

-- 1. brand (20개)

INSERT INTO brand (name) VALUES

('Toyota'), ('Ford'), ('Honda'), ('Chevrolet'), ('Nissan'),
 ('BMW'), ('Mercedes-Benz'), ('Audi'), ('Hyundai'), ('Kia'),
 ('Volkswagen'), ('Subaru'), ('Mazda'), ('Lexus'), ('Volvo'),
 ('Porsche'), ('Jeep'), ('Tesla'), ('Albatross Motors'), ('Sogang Auto');

-- 2. model (20개)

INSERT INTO model (name, year, base_price, brand_id) VALUES

('Camry', 2025, 25000.00, 1), ('Mustang', 2025, 30000.00, 2),
 ('Civic', 2025, 22000.00, 3), ('Malibu', 2024, 23000.00, 4),
 ('Altima', 2024, 24000.00, 5), ('X5', 2026, 60000.00, 6),
 ('C-Class', 2025, 45000.00, 7), ('A4', 2026, 40000.00, 8),
 ('Sonata', 2025, 24000.00, 9), ('K5', 2025, 24500.00, 10),

```
('Golf', 2024, 23000.00, 11), ('Outback', 2025, 28000.00, 12),  
('CX-5', 2025, 27000.00, 13), ('RX', 2026, 50000.00, 14),  
('XC90', 2025, 55000.00, 15), ('911 Carrera', 2026, 105000.00, 16),  
('Wrangler', 2025, 35000.00, 17), ('Model 3', 2025, 40000.00, 18),  
('Nova', 2026, 52000.00, 19), ('T-Express', 2026, 38000.00, 20);
```

-- 3. model_body_style (20개) - Query 5 테스트를 위해 convertible 포함

```
INSERT INTO model_body_style (model_id, body_style) VALUES  
(1, 'Sedan'), (2, 'Coupe'), (2, 'convertible'), (3, 'Sedan'), (4, 'Sedan'),  
(5, 'Sedan'), (6, 'SUV'), (7, 'Sedan'), (7, 'convertible'), (8, 'Sedan'),  
(9, 'Sedan'), (10, 'Sedan'), (11, 'Hatchback'), (12, 'Wagon'), (13, 'SUV'),  
(14, 'SUV'), (15, 'SUV'), (16, 'Coupe'), (16, 'convertible'), (17, 'SUV');
```

-- 4. option (20개)

```
INSERT INTO `option` (color, engine_type, transmission_type) VALUES  
('Red', 'V6', 'Auto'), ('Blue', 'V4', 'Manual'), ('Black', 'V8', 'Auto'),  
('White', 'Electric', '1-Speed'), ('Silver', 'V4', 'Auto'), ('Grey', 'V6', 'Manual'),  
('Yellow', 'V8', 'Auto'), ('Green', 'Hybrid', 'CVT'), ('Red', 'V4', 'Auto'),  
('Blue', 'V6', 'Auto'), ('Black', 'Electric', '1-Speed'), ('White', 'V8', 'Manual'),  
('Silver', 'Hybrid', 'CVT'), ('Grey', 'V4', 'Manual'), ('Red', 'V6', 'Auto'),  
('Blue', 'V8', 'Auto'), ('Black', 'V4', 'CVT'), ('White', 'V6', 'Manual'),  
('Silver', 'V8', 'Auto'), ('Grey', 'Electric', '1-Speed');
```

-- 5. plant (20개)

```
INSERT INTO plant (name, location) VALUES  
('Seoul Plant 1', 'Mapo-gu, Seoul'), ('Seoul Plant 2', 'Gangnam-gu, Seoul'),
```

('Busan Plant', 'Busan'), ('Incheon Plant', 'Incheon'), ('Daegu Plant', 'Daegu'),
('Gwangju Plant', 'Gwangju'), ('Daejeon Plant', 'Daejeon'), ('Ulsan Plant', 'Ulsan'),
('Sejong Plant', 'Sejong'), ('Suwon Plant', 'Suwon'), ('Changwon Plant', 'Changwon'),
('Goyang Plant', 'Goyang'), ('Yongin Plant', 'Yongin'), ('Cheongju Plant', 'Cheongju'),
('Jeonju Plant', 'Jeonju'), ('Cheonan Plant', 'Cheonan'), ('Gimhae Plant', 'Gimhae'),
('Pohang Plant', 'Pohang'), ('Jinju Plant', 'Jinju'), ('Jeju Plant', 'Jeju');

-- 6. dealer (20개)

INSERT INTO dealer (name, address) VALUES

('Sinchon Motors', 'Sinchon-ro, Seoul'), ('Gangnam Auto', 'Gangnam-daero, Seoul'),
('Busan Dealership', 'Haeundae, Busan'), ('Incheon Cars', 'Bupyeong, Incheon'),
('Daegu Auto', 'Jung-gu, Daegu'), ('Gwangju Motors', 'Seo-gu, Gwangju'),
('Daejeon Dealership', 'Yuseong-gu, Daejeon'), ('Ulsan Cars', 'Nam-gu, Ulsan'),
('Sejong Auto', 'Sejong-si'), ('Suwon Motors', 'Paldal-gu, Suwon'),
('Changwon Dealership', 'Seongsan-gu, Changwon'), ('Goyang Cars', 'Ilsan, Goyang'),
('Yongin Auto', 'Giheung-gu, Yongin'), ('Cheongju Motors', 'Heungdeok-gu, Cheongju'),
('Jeonju Dealership', 'Wansan-gu, Jeonju'), ('Cheonan Cars', 'Dongnam-gu, Cheonan'),
('Gimhae Auto', 'Gimhae-si'), ('Pohang Motors', 'Nam-gu, Pohang'),
('Jinju Dealership', 'Jinju-si'), ('Jeju Cars', 'Jeju-si');

-- 7. vehicle (20개) - VIN은 반드시 17자리

INSERT INTO vehicle (VIN, manufacturing_date, current_status, model_id, plant_id,
assemble_date, dealer_id, arrival_date) VALUES

('VIN0000000000000001', '2024-01-10', 'Inventory', 1, 1, '2024-01-15', 1, '2024-01-20'),
('VIN0000000000000002', '2024-02-11', 'Inventory', 2, 2, '2024-02-16', 2, '2024-02-21'),
('VIN0000000000000003', '2024-03-12', 'Inventory', 3, 3, '2024-03-17', 3, '2024-03-22'),

```
( 'VIN0000000000000004', '2024-04-13', 'Inventory', 4, 4, '2024-04-18', 4, '2024-04-23'),
( 'VIN0000000000000005', '2024-05-14', 'Inventory', 5, 5, '2024-05-19', 5, '2024-05-24'),
( 'VIN0000000000000006', '2024-06-15', 'Inventory', 6, 6, '2024-06-20', 6, '2024-06-25'),
( 'VIN0000000000000007', '2024-07-16', 'Inventory', 7, 7, '2024-07-21', 7, '2024-07-26'),
( 'VIN0000000000000008', '2024-08-17', 'Inventory', 8, 8, '2024-08-22', 8, '2024-08-27'),
( 'VIN0000000000000009', '2024-09-18', 'Inventory', 9, 9, '2024-09-23', 9, '2024-09-28'),
( 'VIN0000000000000010', '2024-10-19', 'Inventory', 10, 10, '2024-10-24', 10, '2024-10-29'),
( 'VIN0000000000000011', '2025-01-20', 'Inventory', 11, 11, '2025-01-25', 11, '2025-01-30'),
( 'VIN0000000000000012', '2025-02-21', 'Inventory', 12, 12, '2025-02-26', 12, '2025-03-02'),
( 'VIN0000000000000013', '2025-03-22', 'Inventory', 13, 13, '2025-03-27', 13, '2025-04-01'),
( 'VIN0000000000000014', '2025-04-23', 'Inventory', 14, 14, '2025-04-28', 14, '2025-05-03'),
( 'VIN0000000000000015', '2025-05-24', 'Inventory', 15, 15, '2025-05-29', 15, '2025-06-03'),
( 'VIN0000000000000016', '2025-06-25', 'Inventory', 16, 16, '2025-06-30', 16, '2025-07-05'),
( 'VIN0000000000000017', '2025-07-26', 'Inventory', 17, 1, '2025-07-31', 17, '2025-08-05'),
( 'VIN0000000000000018', '2026-01-27', 'Inventory', 18, 2, '2026-02-01', 18, '2026-02-06'),
( 'VIN0000000000000019', '2026-02-28', 'Inventory', 19, 3, '2026-03-05', 19, '2026-03-10'),
( 'VIN0000000000000020', '2026-03-01', 'Inventory', 20, 4, '2026-03-06', 20, '2026-03-11');
```

-- 8. vehicle_option (20개)

```
INSERT INTO vehicle_option (VIN, option_id) VALUES
```

```
( 'VIN0000000000000001', 1), ( 'VIN0000000000000002', 2), ( 'VIN0000000000000003', 3),
( 'VIN0000000000000004', 4), ( 'VIN0000000000000005', 5), ( 'VIN0000000000000006', 6),
( 'VIN0000000000000007', 7), ( 'VIN0000000000000008', 8), ( 'VIN0000000000000009', 9),
( 'VIN0000000000000010', 10), ( 'VIN0000000000000011', 11), ( 'VIN0000000000000012', 12),
( 'VIN0000000000000013', 13), ( 'VIN0000000000000014', 14), ( 'VIN0000000000000015', 15),
( 'VIN0000000000000016', 16), ( 'VIN0000000000000017', 17), ( 'VIN0000000000000018', 18),
```

```
('VIN000000000000019', 19), ('VIN000000000000020', 20);
```

```
-- 9. customer (20개)
```

```
INSERT INTO customer (name, address, phone, gender, annual_income) VALUES
```

```
('Yubin Jang', 'Mapo-gu, Seoul', '010-1234-5678', 'Female', 45000.00),
```

```
('Seongwon Jeong', 'Gangnam-gu, Seoul', '010-2345-6789', 'Male', 120000.00),
```

```
('Alice Smith', 'Busan', '010-3456-7890', 'Female', 35000.00),
```

```
('Bob Johnson', 'Incheon', '010-4567-8901', 'Male', 55000.00),
```

```
('Charlie Brown', 'Daegu', '010-5678-9012', 'Male', 42000.00),
```

```
('Diana Prince', 'Gwangju', '010-6789-0123', 'Female', 85000.00),
```

```
('Evan Wright', 'Daejeon', '010-7890-1234', 'Male', 28000.00),
```

```
('Fiona Gallagher', 'Ulsan', '010-8901-2345', 'Female', 31000.00),
```

```
('George Miller', 'Sejong', '010-9012-3456', 'Male', 95000.00),
```

```
('Hannah Abbott', 'Suwon', '010-0123-4567', 'Female', 48000.00),
```

```
('Ian Malcolm', 'Changwon', '010-1111-2222', 'Male', 72000.00),
```

```
('Julia Roberts', 'Goyang', '010-2222-3333', 'Female', 61000.00),
```

```
('Kevin Hart', 'Yongin', '010-3333-4444', 'Male', 39000.00),
```

```
('Laura Dern', 'Cheongju', '010-4444-5555', 'Female', 53000.00),
```

```
('Michael Scott', 'Jeonju', '010-5555-6666', 'Male', 46000.00),
```

```
('Nancy Drew', 'Cheonan', '010-6666-7777', 'Female', 34000.00),
```

```
('Oscar Isaac', 'Gimhae', '010-7777-8888', 'Male', 67000.00),
```

```
('Pam Beesly', 'Pohang', '010-8888-9999', 'Female', 29000.00),
```

```
('Quinn Fabray', 'Jinju', '010-9999-0000', 'Female', 78000.00),
```

```
('Ryan Gosling', 'Jeju', '010-0000-1111', 'Male', 88000.00);
```

```
-- 10. sales (20개) - 데이터 삽입 시 트리거 작동(vehicle 상태 'Sold'로 변경)
```

```

INSERT INTO sales (date, sale_price, payment_method, VIN, dealer_id, customer_id) VALUES
('2024-02-15', 26000.00, 'Credit', 'VIN0000000000000001', 1, 1),
('2024-03-20', 31000.00, 'Cash', 'VIN0000000000000002', 2, 2),
('2024-04-10', 23000.00, 'Lease', 'VIN0000000000000003', 3, 3),
('2024-05-05', 24000.00, 'Credit', 'VIN0000000000000004', 4, 4),
('2024-06-18', 25000.00, 'Cash', 'VIN0000000000000005', 5, 5),
('2024-07-22', 62000.00, 'Credit', 'VIN0000000000000006', 6, 6),
('2024-08-30', 46000.00, 'Lease', 'VIN0000000000000007', 7, 7),
('2024-09-15', 41000.00, 'Credit', 'VIN0000000000000008', 8, 8),
('2024-10-10', 25000.00, 'Cash', 'VIN0000000000000009', 9, 9),
('2024-11-05', 25500.00, 'Credit', 'VIN0000000000000010', 10, 10),
('2025-02-15', 24000.00, 'Lease', 'VIN0000000000000011', 11, 11),
('2025-03-20', 29000.00, 'Credit', 'VIN0000000000000012', 12, 12),
('2025-04-10', 28000.00, 'Cash', 'VIN0000000000000013', 13, 13),
('2025-05-05', 52000.00, 'Credit', 'VIN0000000000000014', 14, 14),
('2025-06-18', 56000.00, 'Lease', 'VIN0000000000000015', 15, 15),
('2025-08-22', 106000.00, 'Credit', 'VIN0000000000000016', 16, 16),
('2025-09-15', 36000.00, 'Cash', 'VIN0000000000000017', 17, 17),
('2026-03-01', 41000.00, 'Credit', 'VIN0000000000000018', 18, 18),
('2026-04-10', 53000.00, 'Lease', 'VIN0000000000000019', 19, 19),
('2026-05-05', 39000.00, 'Credit', 'VIN0000000000000020', 20, 20);

```

-- 11. supplier (20개) - Query 2 테스트를 위해 Getrag 포함

```

INSERT INTO supplier (name, contact_information) VALUES
('Getrag', 'contact@getrag.com'), ('Bosch', 'bosch@bosch.com'),
('Denso', 'denso@denso.com'), ('Magna', 'info@magna.com'),

```

('Aisin', 'contact@aisin.com'), ('Continental', 'info@continental.com'),
('ZF Friedrichshafen', 'zf@zf.com'), ('Valeo', 'valeo@valeo.com'),
('Lear', 'lear@lear.com'), ('Faurecia', 'faurecia@faurecia.com'),
('Hyundai Mobis', 'mobis@mobis.com'), ('Mando', 'mando@mando.com'),
('HL Klemmove', 'klemmove@hl.com'), ('LG Innotek', 'innotek@lg.com'),
('Samsung SDI', 'sdi@samsung.com'), ('SK On', 'skon@sk.com'),
('Pirelli', 'pirelli@pirelli.com'), ('Michelin', 'michelin@michelin.com'),
('Bridgestone', 'bridgestone@bridgestone.com'), ('Hankook', 'hankook@hankook.com');

-- 12. supplier_plant (20개) - Query 2 테스트를 위해 Getrag_Plant_1 포함

INSERT INTO supplier_plant (supplier_id, name, location) VALUES

(1, 'Getrag_Plant_1', 'Germany'), (1, 'Getrag_Plant_2', 'USA'),
(2, 'Bosch_Plant_A', 'Germany'), (3, 'Denso_Plant_1', 'Japan'),
(4, 'Magna_Plant_1', 'Canada'), (5, 'Aisin_Plant_A', 'Japan'),
(6, 'Conti_Plant', 'Germany'), (7, 'ZF_Plant_1', 'Germany'),
(8, 'Valeo_Plant', 'France'), (9, 'Lear_Plant', 'USA'),
(10, 'Faurecia_Plant', 'France'), (11, 'Mobis_Ulsan', 'Ulsan, Korea'),
(12, 'Mando_Pyeongtaek', 'Pyeongtaek, Korea'), (13, 'HL_Plant', 'Korea'),
(14, 'LG_Plant', 'Gumi, Korea'), (15, 'SDI_Plant', 'Cheonan, Korea'),
(16, 'SK_Plant', 'Seosan, Korea'), (17, 'Pirelli_Plant', 'Italy'),
(18, 'Michelin_Plant', 'France'), (19, 'Bridgestone_Plant', 'Japan');

-- 13. part (20개) - Query 2 테스트를 위해 transmission 포함

INSERT INTO part (name) VALUES

('transmission'), ('Engine Block'), ('Brake Caliper'), ('Suspension Spring'),
('Exhaust Pipe'), ('Battery Pack'), ('Alternator'), ('Radiator'),

('Clutch Kit'), ('Drive Axle'), ('Steering Wheel'), ('Airbag Module'),
('Leather Seat'), ('Windshield Glass'), ('LED Headlight'), ('Taillight Assembly'),
('Front Bumper'), ('Rearview Mirror'), ('All-Season Tire'), ('Alloy Wheel');

-- 14. produces_part (20개)

```
INSERT INTO produces_part (model_id, plant_id, part_id, produce_date) VALUES  
(1, 1, 2, '2023-11-01'), (2, 2, 3, '2023-11-02'), (3, 3, 4, '2023-11-03'),  
(4, 4, 5, '2023-11-04'), (5, 5, 6, '2023-11-05'), (6, 6, 7, '2023-11-06'),  
(7, 7, 8, '2023-11-07'), (8, 8, 9, '2023-11-08'), (9, 9, 10, '2023-11-09'),  
(10, 10, 11, '2023-11-10'), (11, 11, 12, '2023-11-11'), (12, 12, 13, '2023-11-12'),  
(13, 13, 14, '2023-11-13'), (14, 14, 15, '2023-11-14'), (15, 15, 16, '2023-11-15'),  
(16, 16, 17, '2023-11-16'), (17, 17, 18, '2023-11-17'), (18, 18, 19, '2023-11-18'),  
(19, 19, 20, '2023-11-19'), (20, 20, 1, '2023-11-20');
```

-- 15. supplies (20개) - Query 2 로직 검증(Getrag_Plant_1에서 2023~2024 사이 공급)

```
INSERT INTO supplies (model_id, supplier_id, supplier_plant_name, part_id, supply_date)  
VALUES  
(1, 1, 'Getrag_Plant_1', 1, '2023-12-01'),  
(2, 1, 'Getrag_Plant_1', 1, '2024-01-15'),  
(3, 2, 'Bosch_Plant_A', 3, '2023-12-03'),  
(4, 3, 'Denso_Plant_1', 7, '2023-12-04'),  
(5, 4, 'Magna_Plant_1', 18, '2023-12-05'),  
(6, 5, 'Aisin_Plant_A', 1, '2023-12-06'),  
(7, 6, 'Conti_Plant', 19, '2023-12-07'),  
(8, 7, 'ZF_Plant_1', 1, '2023-12-08'),  
(9, 8, 'Valeo_Plant', 15, '2023-12-09'),
```


(10, 9, 'Lear_Plant', 13, '2023-12-10'),
(11, 10, 'Faurecia_Plant', 5, '2023-12-11'),
(12, 11, 'Mobis_Ulsan', 12, '2023-12-12'),
(13, 12, 'Mando_Pyeongtaek', 3, '2023-12-13'),
(14, 13, 'HL_Plant', 11, '2023-12-14'),
(15, 14, 'LG_Plant', 15, '2023-12-15'),
(16, 15, 'SDI_Plant', 6, '2023-12-16'),
(17, 16, 'SK_Plant', 6, '2023-12-17'),
(18, 17, 'Pirelli_Plant', 19, '2023-12-18'),
(19, 18, 'Michelin_Plant', 19, '2023-12-19'),
(20, 19, 'Bridgestone_Plant', 19, '2023-12-20');

각 테이블마다 20개의 레코드를 가지도록 설정하였습니다.

추가로 제가 만든 7개의 Query는 다음과 같습니다.

```

-- 최근 3년간 다양한 브랜드의 판매 동향을 연, 월, 주, 구매자 성별, 소득 수준별로 파악함.
SELECT
    b.name AS brand_name,
    YEAR(s.date) AS sales_year,
    MONTH(s.date) AS sales_month,
    WEEK(s.date) AS sales_week,
    c.gender,
    CASE
        WHEN c.annual_income < 30000 THEN 'Low'
        WHEN c.annual_income BETWEEN 30000 AND 60000 THEN 'Middle'
        ELSE 'High'
    END AS income_range,
    COUNT(s.sales_id) AS total_sales_volume,
    SUM(s.sale_price) AS total_sales_revenue
FROM sales s
JOIN vehicle v ON s.VIN = v.VIN
JOIN model m ON v.model_id = m.model_id
JOIN brand b ON m.brand_id = b.brand_id
JOIN customer c ON s.customer_id = c.customer_id
WHERE s.date >= DATE_SUB(CURDATE(), INTERVAL 3 YEAR)
GROUP BY b.brand_id, b.name, sales_year, sales_month, sales_week, c.gender, income_range
ORDER BY brand_name, sales_year, sales_month, sales_week;

```

-- 특정 공급사(Getrag)의 특정 공장(Getrag_Plant_1)에서 지정된 날짜 사이에 납품된 변속기(transmission)가 탑재된 차량의 VIN과 해당 구매 고객 정보 탐색함.

```

SELECT
    v.VIN,
    c.name AS customer_name,
    c.phone AS customer_phone
FROM supplier s
JOIN supplier_plant spl ON s.supplier_id = spl.supplier_id
JOIN supplies sp ON sp.supplier_id = spl.supplier_id AND sp.supplier_plant_name = spl.name
JOIN part p ON sp.part_id = p.part_id
JOIN model m ON sp.model_id = m.model_id
JOIN vehicle v ON m.model_id = v.model_id
JOIN sales sl ON v.VIN = sl.VIN
JOIN customer c ON sl.customer_id = c.customer_id
WHERE s.name = 'Getrag'
    AND p.name = 'transmission'
    AND spl.name = 'Getrag_Plant_1'
    AND sp.supply_date BETWEEN '2023-01-01' AND '2024-12-31';

```

-- 최근 1년 동안 매출액 기준 상위 2개 브랜드를 구해줍니다.

```
SELECT
    b.name AS brand_name,
    SUM(s.sale_price) AS total_revenue
FROM sales s
JOIN vehicle v ON s.VIN = v.VIN
JOIN model m ON v.model_id = m.model_id
JOIN brand b ON m.brand_id = b.brand_id
WHERE s.date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)
GROUP BY b.brand_id, b.name
ORDER BY total_revenue DESC
LIMIT 2;
```

-- 최근 1년 동안 판매 대수 기준 상위 2개 브랜드를 구해줍니다.

```
SELECT
    b.name AS brand_name,
    COUNT(s.sales_id) AS total_unit_sales
FROM sales s
JOIN vehicle v ON s.VIN = v.VIN
JOIN model m ON v.model_id = m.model_id
JOIN brand b ON m.brand_id = b.brand_id
WHERE s.date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)
GROUP BY b.brand_id, b.name
ORDER BY total_unit_sales DESC
LIMIT 2;
```

-- 특정 차체 스타일(convertible)이 가장 잘 팔리는 달을 탐색해줌.

```
SELECT
    MONTH(s.date) AS best_selling_month,
    COUNT(s.sales_id) AS total_unit_sales
FROM sales s
JOIN vehicle v ON s.VIN = v.VIN
JOIN model m ON v.model_id = m.model_id
JOIN model_body_style mbs ON m.model_id = mbs.model_id
WHERE mbs.body_style = 'convertible'
GROUP BY best_selling_month
ORDER BY total_unit_sales DESC
LIMIT 1;
```

-- 차량을 평균적으로 가장 오래 보관하고 있는 딜러를 탐색해줌.

```
SELECT
    d.dealer_id,
    d.name,
    AVG(DATEDIFF(s.date, v.arrival_date)) AS avg_inventory_time_days
FROM dealer d
JOIN vehicle v ON d.dealer_id = v.dealer_id
JOIN sales s ON v.VIN = s.VIN
GROUP BY d.dealer_id, d.name
ORDER BY avg_inventory_time_days DESC
LIMIT 1;
```

-- 모든 브랜드에 걸쳐 가장 많은 종류의 모델(distinct model)에 부품을 공급하는 1순위 납품업체 탐색을 해
줌.

```
SELECT
    s.supplier_id,
    s.name,
    COUNT(DISTINCT sp.model_id) AS distinct_model_count
FROM supplier s
JOIN supplier_plant spl ON s.supplier_id = spl.supplier_id
JOIN supplies sp ON sp.supplier_id = spl.supplier_id AND sp.supplier_plant_name = spl.name
GROUP BY s.supplier_id, s.name
ORDER BY distinct_model_count DESC
LIMIT 1;
```

4. Application Development

이제 제가 구현한 main.c 애플리케이션 코드를 간략하게 리뷰해보도록 하겠습니다.

더 자세한 설명은 main.c에서 확인할 수 있으실 겁니다.

```
// 데이터베이스 연결 정보 설정 (본인의 환경에 맞게 수정하세요)
#define DB_HOST "localhost"
#define DB_USER "root"
#define DB_PASS "6339" // 실제 MySQL 비밀번호로 변경
#define DB_NAME "auto_company"
#define DB_PORT 3306
```

먼저, 데이터베이스 연결 정보를 설정해줍니다.

참고로, 제 MySQL Workbench의 비밀번호는 6339입니다.

그리고, 제가 사용하고 있는 database의 name은 auto_company입니다.

```
// 에러 발생 시 처리하는 헬퍼 함수
void finish_with_error(MYSQL *conn)
{
    fprintf(stderr, "\n[Database Error] %s\n", mysql_error(conn));
    mysql_close(conn);
    exit(1);
}
```

다음은 연결 에러와 같은 에러가 발생했을 때,
처리하는 헬퍼 함수입니다.

에러 출력문을 stderr로 출력한 다음,

Mysql_close(conn)을 통해 database 연결을 끊어냅니다.

그리고, exit(1)으로 종료해줍니다.

```

// 쿼리를 실행하고 결과를 표 형태로 출력하는 공통 함수
void execute_and_print_query(MYSQL *conn, const char *query, const char *title)
{
    printf("\n===== \n");
    printf(" %s\n", title);
    printf("===== \n");

    if (mysql_query(conn, query))
    {
        fprintf(stderr, "Query Execution Failed: %s\n", mysql_error(conn));
        return;
    }

    MYSQL_RES *result = mysql_store_result(conn);
    if (result == NULL)
    {
        if (mysql_field_count(conn) == 0)
        {
            // 결과셋이 없는 쿼리(INSERT, UPDATE 등)의 경우
            printf("Query executed successfully. %llu rows affected.\n", mysql_affected_rows(conn));
        }
        else
        {
            fprintf(stderr, "Result Store Failed: %s\n", mysql_error(conn));
        }
        return;
    }

    int num_fields = mysql_num_fields(result);
    MYSQL_ROW row;
    MYSQL_FIELD *field;

    // 1. 헤더(컬럼명) 출력
    while ((field = mysql_fetch_field(result)))
    {
        printf("%-20s ", field->name);
    }

```



```

printf("\n");
for (int i = 0; i < num_fields; i++)
{
    printf("----- ");
}
printf("\n");

// 2. 데이터(행) 출력
int row_count = 0;
while ((row = mysql_fetch_row(result)))
{
    for (int i = 0; i < num_fields; i++)
    {
        printf("%-20s ", row[i] ? row[i] : "NULL");
    }
    printf("\n");
    row_count++;
}

if (row_count == 0)
{
    printf("No records found matching the criteria.\n");
}
else
{
    printf("\nTotal %d row(s) returned.\n", row_count);
}

// 3. 메모리 해제
mysql_free_result(result);
}

```

위의 `execute_and_print_query()` 함수는 해당 쿼리를 실행하고, 표 형태로 출력해주는 공통 함수라고 보시면 됩니다.

```
// ===== [ 7가지 핵심 쿼리 함수 ] =====

void run_query1_sales_trends(MYSQL *conn) ...

void run_query2_defective_part(MYSQL *conn) ...

void run_query3_top_brands_revenue(MYSQL *conn) ...

void run_query4_top_brands_units(MYSQL *conn) ...

void run_query5_seasonal_sales(MYSQL *conn) ...

void run_query6_dealer_inventory(MYSQL *conn) ...

void run_query7_supplier_coverage(MYSQL *conn) ...
```

위의 함수들은 명세서에 주어진 7개의 쿼리를 실행해주는 함수입니다.

내부적으로, `execute_and_print_query()`를 호출하는 메커니즘입니다.

다음은 콘솔에 메뉴바를 출력해주는 함수인 `display_menu()` 함수입니다.

```
void display_menu()
{
    printf("\n");
    printf("=====\n");
    printf("      Automobile Company Database Dashboard      \n");
    printf("=====\n");
    printf("  1. Sales Trends (Past 3 Years)\n");
    printf("  2. Defective Part Tracking\n");
    printf("  3. Top 2 Brands by Revenue\n");
    printf("  4. Top 2 Brands by Unit Sales\n");
    printf("  5. Seasonal Sales Patterns (Convertible)\n");
    printf("  6. Dealer Inventory Efficiency\n");
    printf("  7. Supplier Coverage\n");
    printf("  0. Exit Application\n");
    printf("=====\n");
    printf("Enter your choice (0-7): ");
}
```

다음은 `main()` 부분입니다.

```
// MySQL 초기화 및 연결
MYSQL *conn = mysql_init(NULL);
if (conn == NULL)
{
    fprintf(stderr, "mysql_init() failed\n");
    return EXIT_FAILURE;
}

printf("Connecting to database...\n");
if (mysql_real_connect(conn, DB_HOST, DB_USER, DB_PASS, DB_NAME, DB_PORT, NULL, 0) == NULL)
{
    finish_with_error(conn);
}
printf("Successfully connected to MySQL Database '%s'!\n", DB_NAME);
```

Mysql_init()을 통하여 MySQL을 초기화해줍니다.

그리고, 연결을 시도할 것입니다.

매크로로 정의된 데이터베이스 설정 정보를 인자들로 넘겨주어서, `mysql_real_connect()`를 호출하여 실제 연결을 시도할 것입니다. 만약, 이때 error가 발생한다면, `finish_with_error()`를 호출할 것입니다.

그렇지 않다면, 성공적으로 database와 연결을 수행했다는 의미로 콘솔에 출력문을 출력해줍니다.

```
// 메뉴 루프 실행
int choice = -1;
char input_buffer[100];

while (1)
{
    display_menu();

    // 입력 받기
    if (fgets(input_buffer, sizeof(input_buffer), stdin) != NULL)
    {
        choice = atoi(input_buffer);
    }
    else
    {
        continue;
    }
}
```

```

while (1)
{
    display_menu();

    // 입력 받기
    if (fgets(input_buffer, sizeof(input_buffer), stdin) != NULL)
    {
        choice = atoi(input_buffer);
    }
    else
    {
        continue;
    }

    switch (choice)
    {
        case 1:
            run_query1_sales_trends(conn);
            break;
        case 2:
            run_query2_defective_part(conn);
            break;
        case 3:
            run_query3_top_brands_revenue(conn);
            break;
        case 4:
            run_query4_top_brands_units(conn);
            break;
        case 5:
            run_query5_seasonal_sales(conn);
            break;
        case 6:
            run_query6_dealer_inventory(conn);
            break;
        case 7:
            run_query7_supplier_coverage(conn);
            break;

```

```

            break;
        case 0:
            printf("\nExiting the application. Goodbye!\n");
            mysql_close(conn);
            return EXIT_SUCCESS;
        default:
            printf("\n[Warning] Invalid choice! Please enter a number bet
    }
}

return EXIT_SUCCESS;
}

```

이제부터는 사용자로부터 입력을 받아서(0 ~ 7까지를 입력 받음.) 그에 해당하는 쿼리를 실행해줄 것입니다. 명세서에도 나타난 switch문을 이용하여 구현해보았습니다.

만약 종료를 의미하는 0을 사용자가 입력을 한다면,

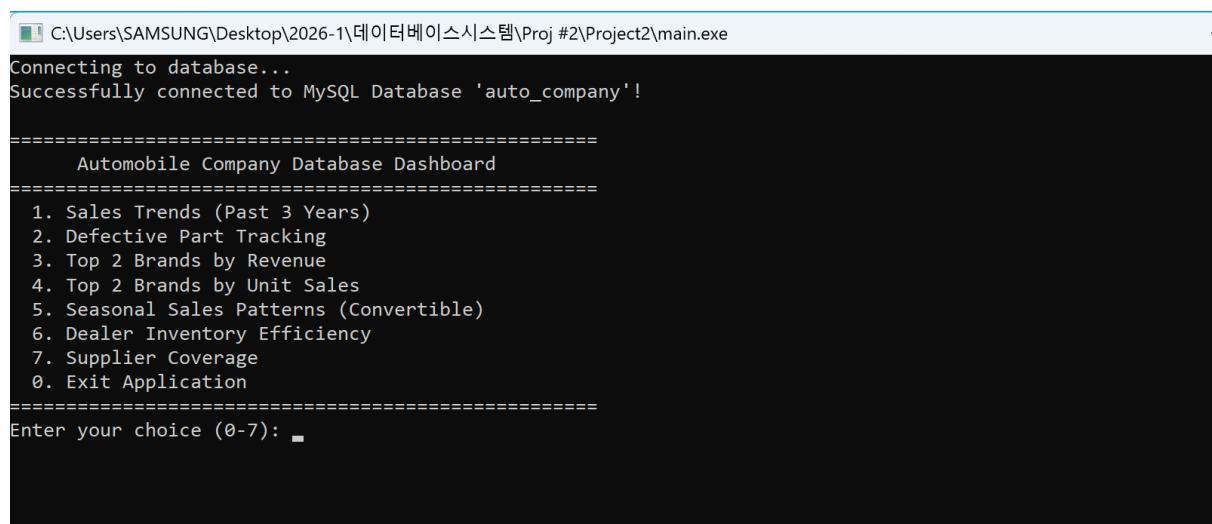
Mysql_close(conn)을 호출하여, database와의 연결을 끊어준 다음,
Return EXIT_SUCCESS로 main()을 종료하게 될 것입니다.

5. Testing and Validation

그렇다면, 이제, 실제 7가지의 쿼리를 실행해보겠습니다.

```
PS C:\Users\SAMSUNG\Desktop\2026-1\데이터베이스시스템\Proj #2\Project2> ./main.exe
```

터미널에서 ./main.exe로 제가 만든 프로그램을 실행시킵니다.



```
C:\Users\SAMSUNG\Desktop\2026-1\데이터베이스시스템\Proj #2\Project2\main.exe
Connecting to database...
Successfully connected to MySQL Database 'auto_company'!

=====
Automobile Company Database Dashboard
=====
1. Sales Trends (Past 3 Years)
2. Defective Part Tracking
3. Top 2 Brands by Revenue
4. Top 2 Brands by Unit Sales
5. Seasonal Sales Patterns (Convertible)
6. Dealer Inventory Efficiency
7. Supplier Coverage
0. Exit Application
=====
Enter your choice (0-7):
```

다음과 같은 화면이 나올 것입니다.

이제, 1번 쿼리를 실행해보겠습니다.

```
=====
Enter your choice (0-7): 1
=====
Query 1: Sales Trends (Past 3 Years)
=====
```

brand_name	sales_year	sales_month	sales_week	gender	income_range	total_sales_volume	total_sales_revenue
--							
Albatross Motors	2026	4	14	Female	High	1	53000.00
Audi	2024	9	37	Female	Middle	1	41000.00
BMW	2024	7	29	Female	High	1	62000.00
Chevrolet	2024	5	18	Male	Middle	1	24000.00
Ford	2024	3	11	Male	High	1	31000.00
Honda	2024	4	14	Female	Middle	1	23000.00
Hyundai	2024	10	40	Male	High	1	25000.00
Jeep	2025	9	37	Male	High	1	36000.00
Kia	2024	11	44	Female	Middle	1	25500.00
Lexus	2025	5	18	Female	Middle	1	52000.00
Mazda	2025	4	14	Male	Middle	1	28000.00
Mercedes-Benz	2024	8	34	Male	Low	1	46000.00
Nissan	2024	6	24	Male	Middle	1	25000.00
Porsche	2025	8	33	Female	Middle	1	106000.00
Sogang Auto	2026	5	18	Male	High	1	39000.00

	brand_name	sales_year	sales_month	sales_week	gender	income_range
▶	Albatross Motors	2026	4	14	Female	High
	Audi	2024	9	37	Female	Middle
	BMW	2024	7	29	Female	High
	Chevrolet	2024	5	18	Male	Middle

잘 출력된 것을 확인할 수 있습니다.

이제, 2번 쿼리를 실행해보겠습니다.

```
=====
Enter your choice (0-7): 2
=====
Query 2: Defective Part Tracking (Getrag Transmissions)
=====
```

VIN	customer_name	customer_phone

VIN000000000000001	Yubin Jang	010-1234-5678
VIN000000000000002	Seongwon Jeong	010-2345-6789

Total 2 row(s) returned.

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
▶	VIN	customer_name	customer_phone
	VIN000000000000001	Yubin Jang	010-1234-5678
	VIN000000000000002	Seongwon Jeong	010-2345-6789

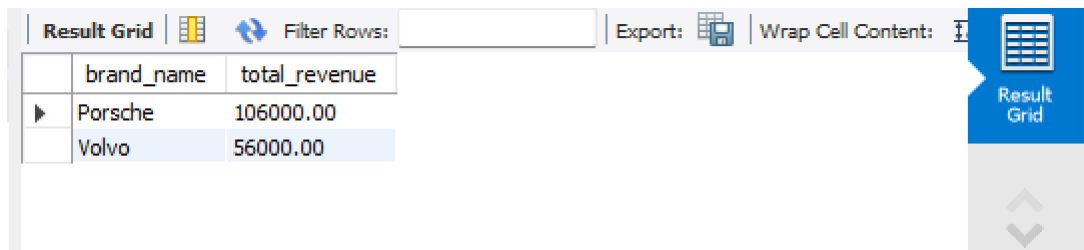
잘 출력된 것을 확인할 수 있습니다.

이제, 3번 쿼리를 실행해보겠습니다.

```
=====
Enter your choice (0-7): 3

=====
Query 3: Top 2 Brands by Revenue (Past 1 Year)
=====
brand_name      total_revenue
-----
Porsche         106000.00
Volvo           56000.00

Total 2 row(s) returned.
```



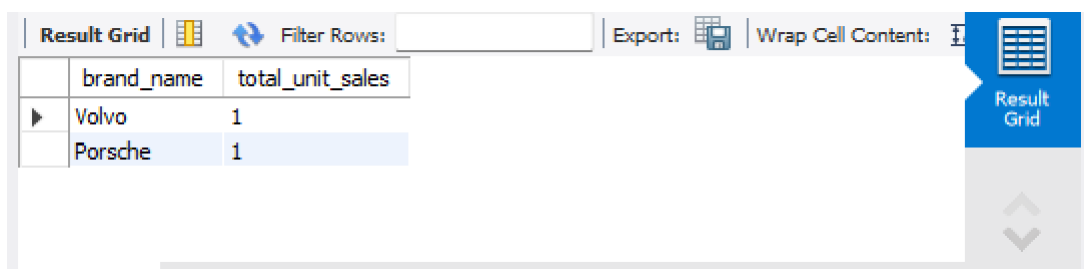
	brand_name	total_revenue
▶	Porsche	106000.00
	Volvo	56000.00

잘 출력된 것을 확인할 수 있습니다.

이제, 4번 쿼리를 실행해보겠습니다.

```
=====
Enter your choice (0-7): 4

=====
Query 4: Top 2 Brands by Unit Sales (Past 1 Year)
=====
brand_name      total_unit_sales
-----
Volvo           1
Porsche         1
```

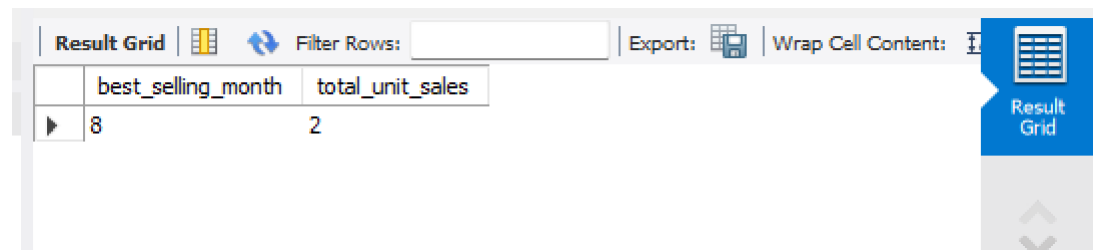


	brand_name	total_unit_sales
▶	Volvo	1
	Porsche	1

잘 출력된 것을 확인할 수 있습니다.

이제, 5번 쿼리를 실행해보겠습니다.

```
=====
Enter your choice (0-7): 5
=====
Query 5: Seasonal Sales Patterns (Convertibles)
=====
best_selling_month    total_unit_sales
-----
8                      2
Total 1 row(s) returned.
```



	best_selling_month	total_unit_sales
▶	8	2

잘 출력된 것을 확인할 수 있습니다.

이제, 6번 쿼리를 실행해보겠습니다.

```
=====
Enter your choice (0-7): 6
=====
Query 6: Dealer Inventory Efficiency (Longest Avg Time)
=====
dealer_id              name                avg_inventory_time_days
-----
20                     Jeju Cars           55.0000
Total 1 row(s) returned.
```


Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Result Grid
	dealer_id	name	avg_inventory_time_days	
▶	20	Jeju Cars	55.0000	

잘 출력된 것을 확인할 수 있습니다.

이제, 7번 쿼리를 한번 실행해보겠습니다.

```
=====
Enter your choice (0-7): 7
=====
Query 7: Supplier Coverage (Most Distinct Models)
=====
supplier_id      name                distinct_model_count
-----
1                Getrag              2
Total 1 row(s) returned.
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Result Grid
	supplier_id	name	distinct_model_count	
▶	1	Getrag	2	

잘 출력된 것을 확인할 수 있습니다.

이제, 종료하기 위해 0을 입력해보겠습니다.

```
0. Exit Application
=====
Enter your choice (0-7): 0
Exiting the application. Goodbye!
```

정상적으로 종료된 것을 확인할 수 있습니다.

이처럼, 제가 만든 7개의 쿼리가 잘 작동하는 것을 확인할 수 있으실 겁니다.

이상입니다.

